

Python AI Engineer – Autonomous Agents – 60-Minute Build Challenge

Time Limit

60 minutes total

Video Length

8–10 minutes

Task

Build a simple autonomous AI agent that can understand a user request, create its own task or TODO list, execute the required steps, and produce a polished Microsoft Word (.docx) document as the final output. You may use any tools, frameworks, or models, but your solution must demonstrate autonomous planning, decision-making, and end-to-end execution.

Requirements

Build a Python API (FastAPI preferred) with `POST /agent` accepting JSON

`{"request": "..."}.` The system should accept a natural language request, determine the tasks required to complete it, generate its own execution plan, execute each step, and return a meaningful response along with the generated Word document. The document may be a proposal, meeting minutes, project plan, business report, technical design, SOP, product specification, or similar structured business document. Use of mock data is allowed where appropriate.

Use a free or locally runnable LLM such as Groq, Gemini (free tier), Grok (free tier), Ollama, LM Studio, or another open-source model so the assignment can be completed without purchasing API credits.

One Real Engineering Improvement (Mandatory)

Implement **ONE** of the following: Conversation memory, Simple RAG, Tool calling, Multi-step planning, Error handling & recovery, Request validation & guardrails, Reflection/self-check, or Retry & fallback logic. Explain what you implemented, why you chose it, and how it improves the agent.

Two Test Inputs

Demonstrate one standard business request and one complex request (ambiguous, multi-step, missing information, conflicting requirements, or requiring the agent to decide its own execution plan and make reasonable assumptions).

Video Submission

Live Demo (3–4 min): Demonstrate the system end-to-end with both test cases, including the agent-generated task list and the final Word document.

What You Built (2–3 min): Explain the architecture, technologies used, agent workflow, planning logic, LLM integration, tool usage, document generation, and API design.

Debugging Insight (1–2 min): Describe one issue faced, the root cause, and how you resolved it.

Tradeoff Discussion (1–2 min): Explain one engineering tradeoff such as Simplicity vs Extensibility, Autonomous Planning vs Deterministic Workflows, Speed vs Functionality, Accuracy vs Development Time, or Single-agent vs Multi-agent architecture.

Rules

Do **NOT** use a fully managed no-code AI platform. You may use OpenAI, Claude, Gemini, Groq, Grok, Ollama, LangChain, LangGraph, CrewAI, MCP, FastAPI, AsyncIO, Flask, or similar tools. Mock data is allowed. Focus on autonomous agent design, engineering decisions, and code quality rather than UI or external system integrations.

Evaluation Criteria

- Python code quality
- Software engineering fundamentals
- Autonomous agent design
- Task planning and reasoning
- Tool orchestration
- API design
- Problem-solving approach
- Debugging ability
- Scalability and architecture thinking
- Ability to explain technical decisions

Why This Assignment

This assignment closely matches the role by testing Python development, autonomous AI agent design, planning and reasoning, tool orchestration, document generation, REST API development, software architecture, debugging, and communication skills while remaining realistically completable within 60 minutes.